



Graphic Era
Deemed to be University

CANDIDATE'S DECLARATION

I hereby certify that the work presented in this project report entitled "Stock Price Prediction System" in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications is a bona fide work carried out by me during the period of January 2026 to May 2026 under the supervision of Dr. Aditya Joshi, Associate Professor, Department of Computer Applications, Graphic Era Deemed to be University, Dehradun, India.

This work has not been submitted elsewhere for the award of a degree/diploma/certificate.

**Name and Signature of
Candidate**

This is to certify that the above-mentioned statement in the candidate's declaration is correct to the best of my knowledge.

Date: _____

Name and Signature of Guide

Signature of Supervisor

Signature of External Examiner

HOD

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled "Stock Price Prediction System", submitted to **Graphic Era University, Dehradun** in partial fulfilment of the requirement for the award of the degree of **MASTER OF COMPUTER APPLICATIONS (MCA)**, is an authentic and original work carried out by Mr. Suryansh Pundir with enrollment number GE-25113598 under my supervision and guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirements of any course of study.

Signature of the Student:

Signature of the Guide:

Date:

Date:

Enrollment No.: GE-25113598

Name and Address of the Student:

Name, Designation and Address of the Guide:

Special Note:

Acknowledgement

First, my sincere and wholehearted gratitude to my project advisor, Professor Dr. Aditya Joshi, for their invaluable guidance, support, and encouragement throughout this project. Their insightful feedback and constructive criticism have significantly contributed to the successful completion of this work. I am deeply grateful for their patience, expertise, and the time they dedicated to mentoring me.

I would also like to extend my sincere thanks to the faculty and staff of the Department of Computer Applications for providing a conducive learning environment and the resources necessary for conducting this research. The academic support and technical assistance offered by the department were instrumental in overcoming the various challenges encountered during this project.

Furthermore, I would like to acknowledge the contributions of the open-source community, whose tools and libraries including yfinance, scikit-learn, FastAPI, and Chart.js were invaluable in the development of this project.

Finally, I would like to thank all those who directly or indirectly contributed to the completion of this project. Your support and assistance have been immensely appreciated.

Suryansh Pundir

Table of Contents

Contents

.....	i
CANDIDATE'S DECLARATION	i
CERTIFICATE OF ORIGINALITY	ii
Acknowledgement	iii
Table of Contents	iv
LIST OF FIGURES	vii
CHAPTER 1: INTRODUCTION	8
1.1 Introduction	8
1.2 Motivation	9
1.3 Objectives	10
1.4 Hardware and Software Requirements	11
1.4.1 Hardware Requirements	11
1.4.2 Software Requirements	12
CHAPTER 2: SYSTEM ANALYSIS & REQUIREMENTS SPECIFICATION	14
2.1 System Analysis	14
1. Problem Identification	14
2. Requirement Gathering	14
3. Feasibility Study	14
4. Dataset Analysis	15
5. Risk Assessment	15
6. Evaluation Metrics	15
2.2 Requirement Specification	16
1. Functional Requirements	16
2. Non-Functional Requirements	17
2.3 Market Demand	18
2.4 Data Flow Diagram (DFD)	20
2.5 Key Stakeholders and their Needs	21
2.6 Sequence Diagram	22

2.7 System Chart.....	23
.....	23
CHAPTER 3: SYSTEM DESIGN.....	24
3.1 System Architecture.....	24
1. Frontend (Client Layer)	24
2. Backend (Application Layer — FastAPI).....	24
3. Data Layer.....	24
4. Model Layer.....	25
3.2 ER Diagram	26
3.3 Flowchart	27
.....	27
3.4 Design of Solution	28
Frontend Design.....	28
Backend Design	28
3.5 Challenges and Solutions.....	29
CHAPTER 4: PROJECT MANAGEMENT	30
4.1 Project Planning and Scheduling	30
4.1.1 Development Approach	30
4.1.2 Project Plan	30
4.2 Risk Management	31
4.2.1 Risk Identification.....	31
4.2.2 Risk Analysis	32
CHAPTER 5: INPUT DESIGN.....	34
User Interface — Input Section	34
Input Validation	35
CHAPTER 6: OUTPUT DESIGN.....	36
6.1 Stats Panel.....	36
6.2 Chart Output.....	37
6.3 Sample Output (Textual Representation).....	38
CHAPTER 7: SYSTEM TESTING, IMPLEMENTATION & MAINTENANCE.....	39
7.1 System Testing.....	39

1. Functional Testing	39
2. Performance Testing	40
3. Error Handling Testing	40
4. Browser Compatibility Testing.....	41
7.2 Implementation	42
7.3 Maintenance	43
CHAPTER 8: SUMMARY AND FUTURE SCOPE	44
8.1 Summary	44
8.2 Achievements.....	45
8.3 Future Scope	45
REFERENCES / BIBLIOGRAPHY.....	46
1. Research Papers	46
2. Websites and Documentation	47
APPENDICES	48
Appendix A: Glossary.....	48
Appendix B: Project File Structure.....	49
Appendix C: Dependencies (requirements.txt).....	49
Appendix D: How to Run the Project.....	50

LIST OF FIGURES

Figure 2.1 Data Flow Diagram of Stock Price Prediction System	12
Figure 2.2 Sequence Diagram	15
Figure 2.3 System Chart	16
Figure 3.1 ER Diagram	18
Figure 3.2 Flowchart	19
Figure 3.3 User Interaction Flow	20

CHAPTER 1: INTRODUCTION

1.1 Introduction

Stock markets play a vital role in a country's economy, acting as an indicator of the financial health of businesses and industries. Accurately predicting the movement of stock prices is one of the most challenging problems in the domain of financial analysis. The fluctuating nature of stock prices, influenced by company performance, global events, investor sentiment, and macroeconomic conditions, makes prediction a difficult but highly valuable task.

With the rapid advancement of machine learning and data science, there is now a reliable way to analyze historical stock price data and identify patterns that may help forecast future prices. This project, the Stock Price Prediction System, aims to leverage these technologies to create a practical, web-based application that can assist investors and analysts in making more informed decisions.

The system fetches live historical stock data using the `yfinance` library, processes it, and trains a Linear Regression model using `scikit-learn`. The model is then used to predict the next closing price of a given stock. The results are displayed through an interactive web interface built with HTML, CSS, JavaScript, and Chart.js, backed by a FastAPI server.

The project focuses on simplicity, clarity, and practical usability, making it suitable as a demonstration of real-world application of machine learning in finance without relying on overly complex or opaque models.

1.2 Motivation

The motivation behind developing the Stock Price Prediction System comes from several real-world needs and observations:

1. **Financial Decision Making:** Investors, especially retail investors, often lack the tools to analyze large amounts of historical data quickly. A system that automates this process and provides a predicted price can significantly aid decision making.
2. **Application of Machine Learning:** Stock prediction is a classic application of machine learning algorithms. Implementing it provides a solid demonstration of how regression models can be applied to time-series financial data.
3. **Accessibility:** Most advanced trading and prediction tools are either expensive or require expert knowledge. This project provides a free, easy-to-use web interface that anyone can use by simply entering a stock symbol.
4. **Learning and Education:** From an academic perspective, building such a system involves a combination of data fetching, preprocessing, model training, and full-stack web development, making it an excellent learning exercise.
5. **Market Relevance:** With growing interest in algorithmic trading and data-driven investing, tools that assist in predicting stock price trends are increasingly in demand.
6. **Global Stock Coverage:** The use of yfinance allows the system to support stocks from major global exchanges, including Indian stocks from NSE (using .NS suffix), US stocks like AAPL, TSLA, and MSFT, and many others.

1.3 Objectives

The primary objectives of the Stock Price Prediction System are as follows:

1. Develop a functional web application that allows users to enter any valid stock symbol and receive a predicted next-day closing price.
2. Fetch and process real historical stock data using the yfinance Python library.
3. Train a Linear Regression model on recent historical closing prices and use it to make a prediction for the next trading day.
4. Display the results visually through a Chart.js line graph showing recent prices alongside the predicted value.
5. Provide additional insights such as the last available closing price and a trend indicator (UP or DOWN) based on the prediction.
6. Build a clean, user-friendly web interface accessible to non-technical users.
7. Ensure proper error handling for invalid symbols, missing data, and network issues.

1.4 Hardware and Software Requirements

1.4.1 Hardware Requirements

S.N.	Hardware Component	Specification
1.	Computer	A standard desktop or laptop with sufficient processing power to run Python-based machine learning algorithms.
2.	Memory (RAM)	At least 4 GB RAM; 8 GB or more recommended for handling data processing and running the server.
3.	Storage	Minimum 1 GB free disk space for libraries, virtual environment, and project files. SSD recommended for faster access.
4.	Internet Connection	Required for fetching live stock data from Yahoo Finance via the yfinance library.
5.	Display	Any standard monitor to view the web-based user interface.

1.4.2 Software Requirements

S.N.	Software	Specification / Purpose
1.	Operating System	Windows 10/11 or any Linux distribution capable of running Python 3.10+.
2.	Programming Language	Python 3.10+ — used for backend development, data processing, and model training.
3.	Development Environment	Visual Studio Code (VS Code) with Python extension for code editing and debugging.
4.	Web Framework	FastAPI — a modern, high-performance Python web framework used to serve the application and expose API routes.
5.	Data Fetching Library	yfinance — an open-source Python library to download historical stock market data from Yahoo Finance.
6.	Machine Learning Library	scikit-learn — used to implement the Linear Regression model for price prediction.
7.	Data Processing	pandas and NumPy — for data manipulation, feature creation, and array operations.

8.	Frontend	HTML5, CSS3, and Vanilla JavaScript — for building the user interface.
9.	Charting Library	Chart.js (CDN) — for rendering interactive line charts of historical and predicted prices.
10.	Template Engine	Jinja2 — used with FastAPI to serve the HTML frontend as a template.
11.	Server	Uvicorn — an ASGI server used to run the FastAPI application locally.

CHAPTER 2: SYSTEM ANALYSIS & REQUIREMENTS SPECIFICATION

2.1 System Analysis

The system analysis process for a Stock Price Prediction project involves understanding the problem domain, defining system requirements, and planning the overall architecture. The steps involved are described below:

1. Problem Identification

- Understand the domain of financial data analysis and the challenges involved in stock price forecasting.
- Identify the limitations of manual analysis — time-consuming, error-prone, and inaccessible to non-experts.
- Define the system's purpose: provide an automated, easy-to-use interface to predict stock closing prices based on historical data.

2. Requirement Gathering

- Functional: Fetch historical stock data, train a prediction model, return predicted price and recent prices via an API.
- Non-Functional: Fast response, clean UI, error handling, compatibility with major stock exchanges worldwide.

3. Feasibility Study

- Technical Feasibility: All tools used (FastAPI, yfinance, scikit-learn, Chart.js) are open-source and well-documented.
- Economic Feasibility: The entire stack is free and requires no paid services or licenses.

- Operational Feasibility: The system is accessible via a web browser and requires no technical knowledge to use.

4. Dataset Analysis

- Data is fetched dynamically from Yahoo Finance for the last 120 days using yfinance.
- Closing price data is used as the primary feature for the prediction model.
- A sliding window of 5 days is used to create feature-label pairs for training.

5. Risk Assessment

- Network dependency: yfinance requires an active internet connection to fetch data.
- Market volatility: Linear regression may underperform during periods of high market volatility.
- Invalid symbols: Users may enter incorrect or delisted stock symbols — handled via error messages.

6. Evaluation Metrics

- Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) can be used to assess model accuracy.
- Visual comparison of predicted price vs. actual prices serves as qualitative validation.

2.2 Requirement Specification

1. Functional Requirements

- The system shall accept a stock symbol as input from the user (e.g., AAPL, TSLA, RELIANCE.NS).
- The system shall fetch the last 120 days of historical closing price data using the yfinance library.
- The system shall prepare a training dataset using a 5-day lookback window.
- The system shall train a Linear Regression model on this data and predict the next closing price.
- The system shall return the predicted price, last 30 days of prices, and corresponding dates as a JSON response.
- The frontend shall display the predicted price, last close, and trend direction (UP/DOWN).
- The frontend shall render a line chart showing historical prices and the predicted value.
- The system shall display meaningful error messages for invalid symbols or unavailable data.

2. Non-Functional Requirements

- **Performance:** The prediction API should respond within 5–10 seconds, including data fetch and model training time.
- **Usability:** The interface should be simple enough for a non-technical user to operate without instructions.
- **Reliability:** The system should handle exceptions gracefully and not crash on bad input.
- **Scalability:** The backend can be extended to support multiple simultaneous requests with minimal changes.
- **Platform Independence:** The application should run on any modern web browser and operating system.

2.3 Market Demand

The demand for stock prediction and financial analytics tools has increased significantly in recent years due to the growing interest in data-driven decision-making in financial markets. With the availability of online trading platforms and easy access to market data, more individuals and organizations are actively participating in stock trading and investment activities.

This growth is driven by multiple sectors:

- **Retail Investors:** Individual investors increasingly prefer using simple and accessible tools to assist in buy and sell decisions rather than relying completely on brokers or financial advisors. These tools help them understand market trends and make more informed choices.
- **Education and Research:** In academic environments, stock prediction systems serve as practical tools for students and researchers studying finance, economics, and data science. They provide hands-on experience with real-world datasets and help in understanding concepts like trend analysis and predictive modeling.
- **Financial Institutions:** Banks, hedge funds, and trading firms use analytical tools to support their decision-making processes. While large institutions rely on advanced systems, simpler prediction models still play a role in initial analysis and strategy development.
- **FinTech Startups:** The rapid growth of financial technology companies has increased the demand for integrating prediction and analytics features into digital platforms. These tools enhance user experience by providing insights and assisting users in making investment decisions.
- **Portfolio Management:** Investors and fund managers use stock trend data to manage their portfolios effectively. Prediction systems help in identifying potential risks and opportunities, enabling better allocation of assets and improved risk management.

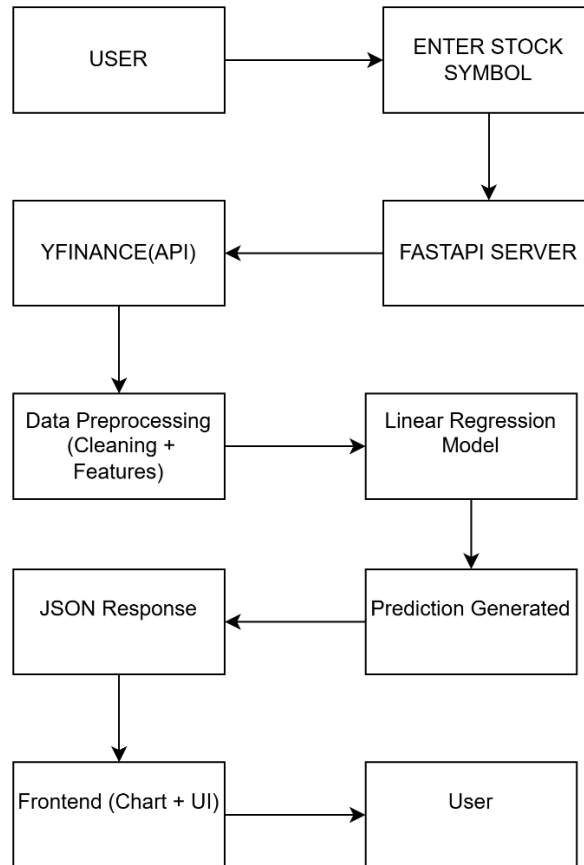
In addition to these sectors, the rise of algorithmic trading and the increasing importance of artificial intelligence in finance further highlight the relevance of such systems. Even

simple prediction models can provide valuable insights when combined with real-time data and visualization tools.

Overall, the growing reliance on technology in financial markets indicates a strong and continuing demand for stock price prediction systems. These tools not only assist in decision-making but also contribute to better understanding and analysis of market behavior.

2.4 Data Flow Diagram (DFD)

The Data Flow Diagram below represents the flow of information through the Stock Price Prediction System, from user input to the displayed output.



[Figure 2.1 — Data Flow Diagram of Stock Price Prediction System]

The DFD shows the following flow: User enters stock symbol → System fetches data from Yahoo Finance via yfinance → Preprocessing and feature creation → Model training using Linear Regression → Prediction generated → JSON response sent to frontend → Chart and stats displayed to user.

2.5 Key Stakeholders and their Needs

- **End Users (Investors/Students):**

End users require a simple and user-friendly interface that allows them to quickly input stock symbols and obtain results. They expect reasonably accurate predictions, clear graphical representations of stock trends, and fast response times. For students, the system should also help in understanding basic concepts of stock analysis and machine learning.

- **Developers:**

Developers need well-structured and modular code that is easy to understand, modify, and extend. Clear API endpoints and proper documentation are important so that new features, such as additional prediction models or improved visualization, can be integrated without difficulty. The system should also be easy to debug and maintain.

- **Academic Institutions:**

Educational institutions require a functional system that demonstrates the practical application of machine learning in finance. The project should clearly show data collection, preprocessing, model training, and prediction, making it suitable for teaching and demonstration purposes.

- **Financial Analysts:**

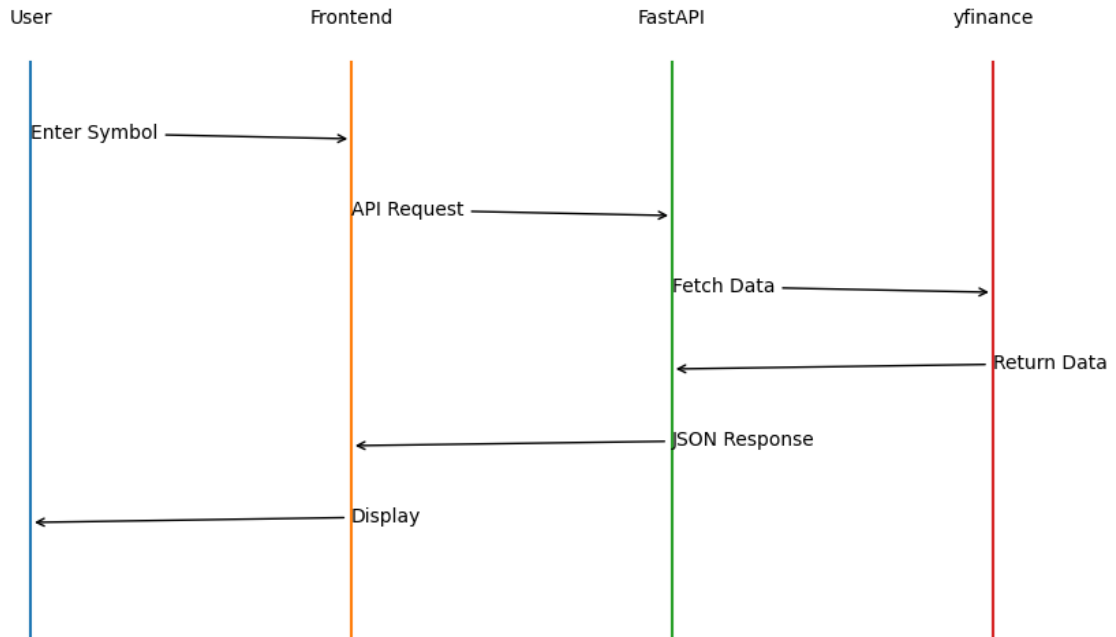
Analysts require access to reliable historical stock data along with meaningful visualizations. The system should provide trend indicators and insights that help in understanding market behavior. While not a replacement for advanced tools, it should assist in basic analysis and decision support.

- **System Administrators:**

Administrators need the system to be easy to deploy and maintain. It should have minimal configuration requirements, stable performance, and proper error handling. Logging mechanisms are important for monitoring system activity and identifying issues when they occur.

2.6 Sequence Diagram

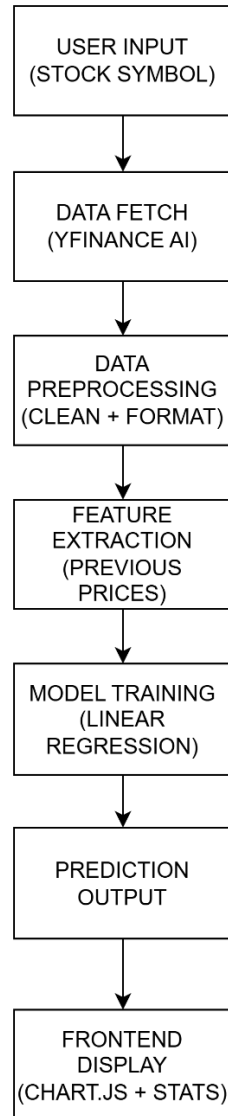
The sequence diagram illustrates the interaction between the user, the frontend, the FastAPI backend, and the Yahoo Finance data source:



[Figure 2.2 — Sequence Diagram]

Steps: (1) User enters stock symbol and clicks Predict. (2) JavaScript sends GET `/predict/{symbol}` to FastAPI. (3) FastAPI calls yfinance to download data. (4) FastAPI processes data, trains model, runs prediction. (5) JSON response returned to frontend. (6) Frontend renders chart and displays prediction stats.

2.7 System Chart



[Figure 2.3 — System Chart]

The system chart shows the complete pipeline: Input (Stock Symbol) → Data Fetch (yfinance) → Preprocessing → Feature Extraction (5-day window) → Linear Regression Training → Prediction Output → Frontend Display (Chart.js + Stats).

CHAPTER 3: SYSTEM DESIGN

This chapter provides a detailed overview of the system design for the Stock Price Prediction project. The design includes system architecture, ER diagrams, flowcharts, and the overall solution design.

3.1 System Architecture

The system is structured as a two-tier web application with the following components:

1. Frontend (Client Layer)

- Built using HTML5, CSS3, and Vanilla JavaScript.
- User enters a stock symbol and clicks Predict.
- JavaScript fetches data from the backend API using the `fetch()` function.
- Chart.js renders a line graph showing the last 30 closing prices and the predicted value.
- Result stats (predicted price, last close, trend) are displayed below the input.

2. Backend (Application Layer — FastAPI)

- Built using Python and FastAPI.
- Route `GET /` serves the frontend HTML page using Jinja2Templates.
- Route `GET /predict/{symbol}` handles all prediction logic.
- Static files (CSS, JS) are served using FastAPI's `StaticFiles` mount.

3. Data Layer

- yfinance fetches historical OHLCV data for the given stock symbol from Yahoo Finance.
- Only the closing price column is used for prediction.

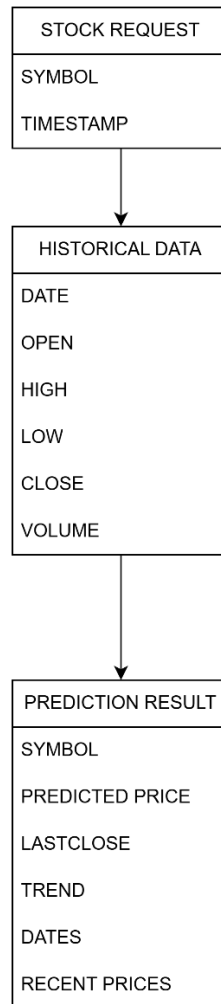
- No persistent database is used; all computation is done in-memory per request.

4. Model Layer

- A sliding window of 5 days of closing prices forms the feature vector (X).
- The next day's closing price is the label (y).
- A LinearRegression model from scikit-learn is trained on this dataset.
- The last 5 available closing prices are fed into the trained model to predict the next price.

3.2 ER Diagram

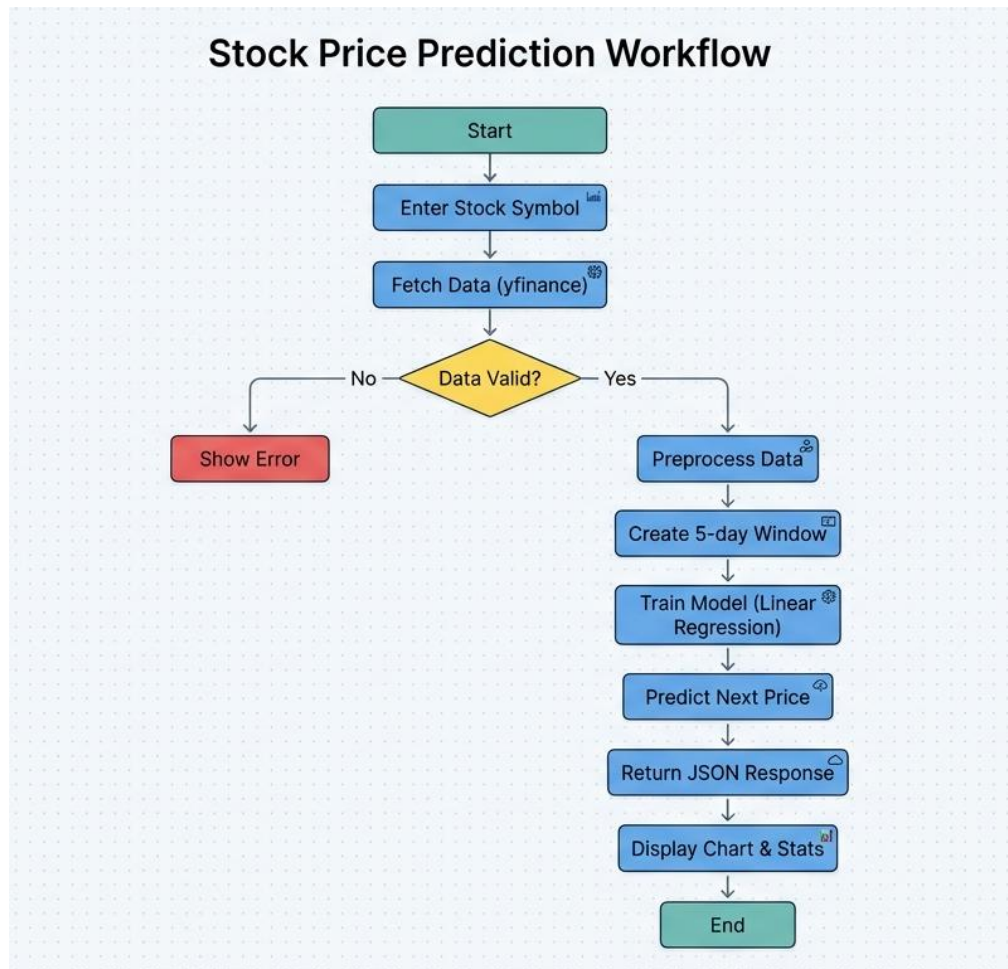
Since this system does not use a persistent database, the ER diagram represents the logical data entities and their relationships within the application:



[Figure 3.1 — ER Diagram]

Entities: StockRequest (symbol, timestamp), HistoricalData (date, open, high, low, close, volume), PredictionResult (symbol, predicted_price, last_close, trend, dates, recent_prices).

3.3 Flowchart



[Figure 3.2 — Flowchart of Prediction Process]

The flowchart follows: Start → User Enters Symbol → Fetch Data from Yahoo Finance → Is data valid? (No → Show Error) → (Yes → Preprocess Data) → Create 5-day Sliding Windows → Train Linear Regression Model → Predict Next Price → Return JSON → Display Chart & Stats → End.

3.4 Design of Solution

Frontend Design

- Centered card layout on a light grey background.
- Input field for stock symbol with a blue Predict button.
- Loading indicator shown during API call.
- Stats section: Predicted Price, Last Close, Trend (UP/DOWN with color).
- Chart.js line graph with two datasets: blue for historical prices, orange for predicted point.

Backend Design

- FastAPI serves both static files and the template HTML page.
- The `/predict/{symbol}` endpoint handles the full data fetch + train + predict pipeline.
- All errors (bad symbol, empty data, library exceptions) are caught and returned as JSON messages.

3.5 Challenges and Solutions

Challenge	Solution
Starlette version incompatibility with TemplateResponse	Changed argument order: TemplateResponse(request, 'index.html') instead of passing request inside context dict.
Invalid or delisted stock symbols entered by user	Wrapped the entire predict function in try/except; returns a JSON error message if data is empty.
Chart not rendering the predicted point correctly	Used spanGaps: false and a null-padded dataset array to separate historical and predicted datasets visually.
Slow response due to model training per request	Linear Regression is extremely fast; training on 100+ data points takes under 1 second, making it acceptable.
CORS issues during frontend-backend communication	Since both are served from the same FastAPI server, CORS is not an issue.

CHAPTER 4: PROJECT MANAGEMENT

4.1 Project Planning and Scheduling

4.1.1 Development Approach

The Agile Methodology was chosen for this project. It allows for iterative development with frequent reviews, making it easy to incorporate feedback and fix issues as they arise. The project was divided into short sprints of one to two weeks each.

4.1.2 Project Plan

Phase	Task	Deliverable	Duration
1	Project Initialization	Project Charter, Scope Definition	1 Week
2	Requirement Gathering	Requirements Document	1 Week
3	System Design	Architecture, Flowcharts, ER Diagram	1 Week
4	Backend Development	FastAPI app, prediction endpoint	2 Weeks
5	Frontend Development	HTML, CSS, JavaScript, Chart.js	1 Week
6	Integration & Testing	Working application, test reports	1 Week
7	Documentation & Submission	Project Report	1 Week

4.2 Risk Management

4.2.1 Risk Identification

- **yfinance API dependency:**

The system relies on the yfinance library, which in turn depends on Yahoo Finance as a data source. Any changes in Yahoo Finance's internal API, structure, or rate limiting policies may affect data retrieval. This can lead to temporary failures or incomplete data, impacting the overall functionality of the system.

- **Model accuracy:**

The system uses a Linear Regression model, which assumes a linear relationship in the data. However, stock markets are highly dynamic and often exhibit non-linear patterns influenced by multiple external factors. As a result, the model may not always produce accurate predictions, especially during volatile market conditions.

- **Insufficient historical data:**

Some stocks, particularly newly listed companies or inactive/delisted symbols, may not have sufficient historical data available. In such cases, the system may not be able to train the model effectively, leading to errors or unreliable predictions.

- **Server errors:**

The prediction process involves multiple steps such as data fetching, preprocessing, and model training. Unexpected issues at any stage—such as invalid inputs, missing values, or runtime exceptions—may cause server-side errors. Without proper handling, this can interrupt the application's functionality.

- **Browser compatibility:**

The frontend uses modern JavaScript features like `fetch()` and visualization libraries such as Chart.js. While these are widely supported in modern browsers, older browsers may not fully support them, which can lead to display issues or reduced functionality.

4.2.2 Risk Analysis

Risk	Probability	Impact	Overall Risk
yfinance API failure	Low	High	Medium
Model inaccuracy	Medium	Medium	Medium
Insufficient data	Medium	High	High
Unhandled server error	Low	High	Medium
Browser incompatibility	Low	Low	Low

4.2.3 Risk Mitigation Strategies

- **For yfinance dependency:**

Since the system relies on the yfinance library to fetch real-time stock data, issues such as network failure or invalid stock symbols may occur. To handle this, the application provides clear and user-friendly error messages, guiding users to check their internet connection or verify the correctness of the entered symbol. This ensures a smoother user experience even when external data sources fail.

- **For model inaccuracy:**

The prediction model used in the system is based on historical data and simple machine learning techniques. As stock markets are highly volatile and influenced by many external factors, the predictions may not always be accurate. Therefore, the system clearly communicates to users that the generated results are only estimates and should not be considered as financial advice.

- **For insufficient data:**

In some cases, the requested stock may not have enough historical data available for effective model training. To address this, the system includes a validation check to ensure that a minimum number of data points (at least 10) are available before proceeding. If this condition is not met, the system displays an appropriate message instead of producing unreliable results.

- **For unhandled errors:**

To improve system reliability, all backend endpoint logic is wrapped within try/except blocks. This prevents the application from crashing due to unexpected errors and ensures that users receive meaningful error messages rather than technical traces. It also helps in easier debugging and maintenance.

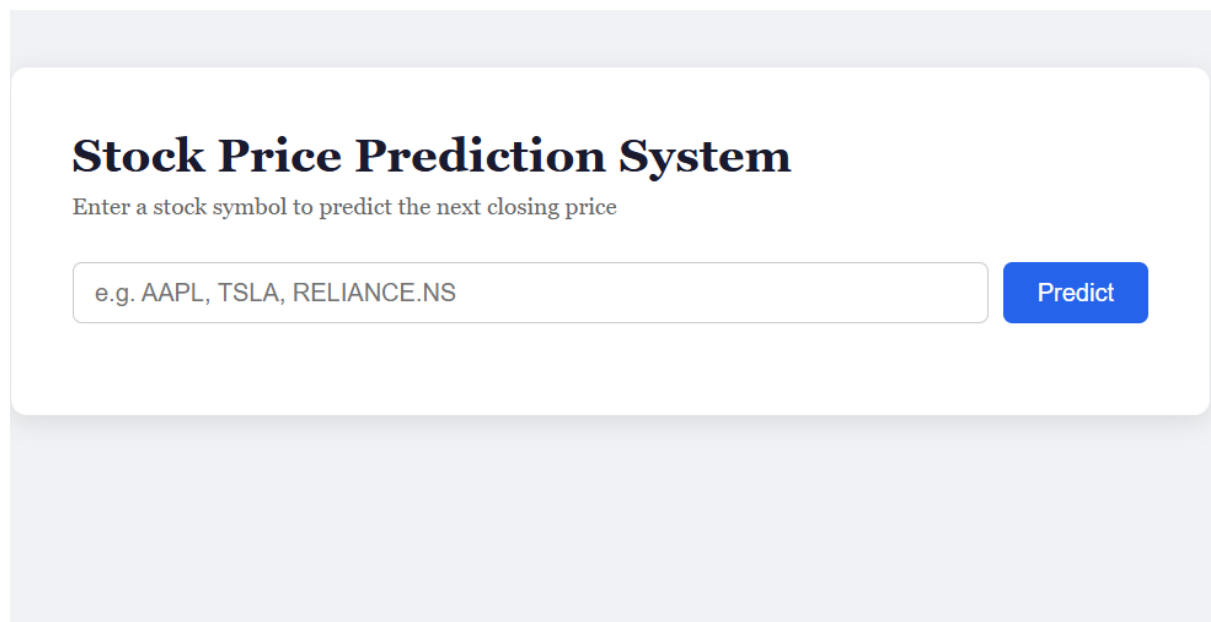
- **For browser compatibility issues:**

The frontend is built using standard web technologies such as HTML, CSS, JavaScript, and Chart.js. These technologies are widely supported across modern browsers released after 2018. This ensures that users can access and use the application smoothly without facing compatibility issues on most devices.

CHAPTER 5: INPUT DESIGN

The input design of the Stock Price Prediction System is centered around simplicity. The user is presented with a single input field where they can type in a valid stock symbol. The interface is designed to minimize friction and require no technical knowledge.

User Interface — Input Section



Stock Price Prediction System

Enter a stock symbol to predict the next closing price

e.g. AAPL, TSLA, RELIANCE.NS

Predict

[Screenshot: Input card with stock symbol field and Predict button]

The input section includes:

- A text input box accepting any valid stock ticker symbol (e.g., AAPL, MSFT, TSLA, RELIANCE.NS, TCS.NS).
- A blue Predict button that triggers the API call.
- A loading message displayed while the backend fetches data and trains the model.

- An error message box in red, shown when the symbol is invalid or data is unavailable.

Users can also press the Enter key instead of clicking the button, as the input field has a keydown event listener attached to it.

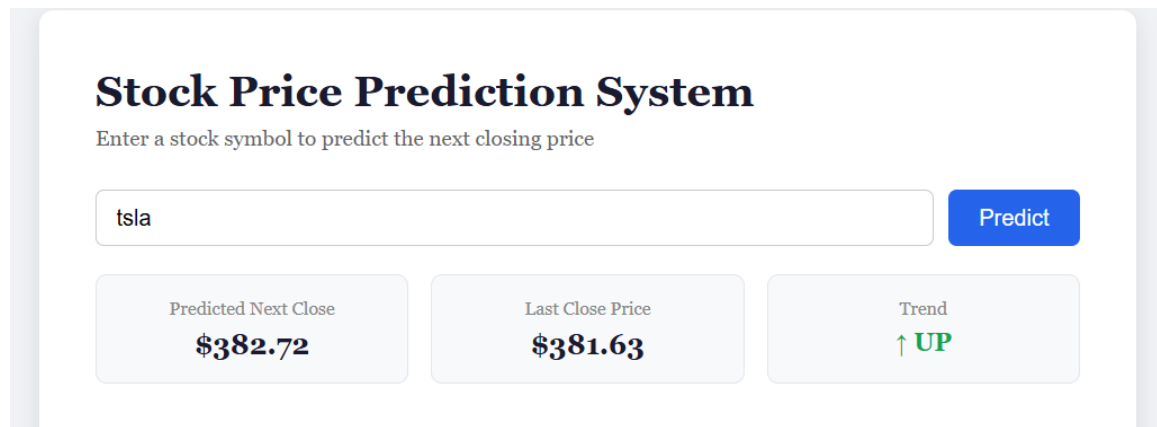
Input Validation

- Empty input: Triggers a browser alert asking the user to enter a symbol.
- Invalid symbol: yfinance returns an empty DataFrame, which the backend detects and responds with an appropriate error message.
- Network failure: A catch block in the JavaScript fetch call displays a network error message to the user.

CHAPTER 6: OUTPUT DESIGN

Once a valid prediction is made, the system displays a rich output to the user including statistical highlights and a visual chart.

6.1 Stats Panel



The screenshot shows a web interface for a 'Stock Price Prediction System'. At the top, the title 'Stock Price Prediction System' is displayed in a bold, dark blue font. Below the title is a subtitle 'Enter a stock symbol to predict the next closing price'. A text input field contains the symbol 'tsla', and a blue 'Predict' button is to its right. Below the input field are three light gray cards. The first card is labeled 'Predicted Next Close' and shows '\$382.72'. The second card is labeled 'Last Close Price' and shows '\$381.63'. The third card is labeled 'Trend' and shows a green upward arrow followed by 'UP'.

Predicted Next Close	Last Close Price	Trend
\$382.72	\$381.63	↑ UP

[Screenshot: Stats panel showing Predicted Price, Last Close, Trend]

The stats panel consists of three cards displayed side by side:

- Predicted Next Close: The price predicted by the Linear Regression model for the next trading day.
- Last Close Price: The most recent available closing price from the historical data.
- Trend: An arrow indicator — green UP arrow if the predicted price is higher than the last close, red DOWN arrow otherwise.

6.2 Chart Output



[Screenshot: Chart.js line graph with historical prices and predicted point]

The Chart.js line graph includes:

- Blue line: Last 30 days of actual closing prices plotted on the x-axis (dates) and y-axis (price in currency).
- Orange point: The predicted next-day closing price plotted at the 'Next Day' position on the chart.
- Legend: Clearly labels both datasets — 'Closing Price' and 'Predicted Price'.
- Y-axis: Price values formatted with a \$ prefix.
- X-axis: Dates formatted to show only the most recent 10 dates for readability.

6.3 Sample Output (Textual Representation)

Output Field	Example Value
Stock Symbol	AAPL
Predicted Next Close	\$211.42
Last Close Price	\$209.87
Trend	↑ UP (green)
Chart	30-day line + predicted orange point
Message	Success

CHAPTER 7: SYSTEM TESTING, IMPLEMENTATION & MAINTENANCE

7.1 System Testing

Testing was carried out at multiple stages to ensure the system functions correctly and handles edge cases gracefully.

1. Functional Testing

TC#	Test Case	Input	Expected Output	Status
TC01	Valid US Stock	AAPL	Predicted price, chart, stats	PASS
TC02	Valid Indian Stock	RELIANCE.NS	Predicted price for NSE stock	PASS
TC03	Invalid Symbol	XYZXYZ123	Error: No data found	PASS
TC04	Empty Input	(blank)	Alert: Enter a symbol	PASS
TC05	Enter Key Trigger	TSLA + Enter	Same as button click	PASS
TC06	Multiple Requests	AAPL then MSFT	Chart updates correctly	PASS
TC07	Long symbol	BAJAJFINSERV.NS	Correct data fetched	PASS

2. Performance Testing

- Response time for AAPL (120 days data, training + prediction): approximately 3–6 seconds on a standard laptop.
- Chart renders within 0.5 seconds of receiving the JSON response.
- Multiple back-to-back requests handled correctly with chart instance properly destroyed and recreated.

3. Error Handling Testing

- Invalid stock symbol: Returns JSON {"message": "No data found for this symbol"}.
- Network disconnected during fetch: JavaScript catch block shows 'Network error' message.
- yfinance exception: Caught by the try/except in the predict endpoint and returned as a readable error.

4. Browser Compatibility Testing

Browser	Version Tested	Status
Google Chrome	Latest	PASS
Mozilla Firefox	Latest	PASS
Microsoft Edge	Latest	PASS
Safari (macOS)	Latest	PASS

7.2 Implementation

The implementation was carried out in the following steps:

- Environment Setup: Python virtual environment created, all dependencies installed using `pip install -r requirements.txt`.
- Project Structure: Created the folders `templates/` and `static/` in the project root, placed `index.html` in `templates/` and `style.css`, `script.js` in `static/`.
- Backend Development: Wrote `app.py` with two routes — `GET /` and `GET /predict/{symbol}`. `StaticFiles` and `Jinja2Templates` configured.
- Frontend Development: Wrote `index.html` with input field, button, loading message, stats panel, and canvas. Wrote `script.js` with fetch logic, `Chart.js` rendering, and error handling. Wrote `style.css` for layout and styling.
- Testing: Tested with multiple stock symbols, verified error handling, confirmed chart renders correctly.
- Bug Fix: Fixed a Starlette version incompatibility in the `TemplateResponse` call.
- Final Run: Application confirmed working at `http://127.0.0.1:8000`.

7.3 Maintenance

- Regular dependency updates: yfinance, scikit-learn, and FastAPI should be updated periodically to avoid compatibility issues.
- Yahoo Finance API changes: If yfinance breaks due to upstream changes, the data fetching logic in app.py may need to be updated.
- Model upgrades: The Linear Regression model can be replaced with Random Forest or Gradient Boosting for improved accuracy.
- Feature additions: Moving averages, RSI, MACD, and other technical indicators can be added as additional features.
- Deployment: For production, the app can be deployed on platforms like Railway, Render, or AWS using Docker.

CHAPTER 8: SUMMARY AND FUTURE SCOPE

8.1 Summary

The Stock Price Prediction System is a full-stack web application that demonstrates the application of machine learning in financial data analysis. The project successfully achieves all its stated objectives — fetching real-time historical stock data, training a regression model, predicting the next closing price, and presenting the results through an interactive web interface.

The key components of the system are:

- FastAPI backend serving both the frontend and the prediction API.
- yfinance for downloading real stock market data from Yahoo Finance.
- scikit-learn LinearRegression model trained on a 5-day sliding window of closing prices.
- Chart.js line graph displaying 30 days of historical prices and the predicted next-day price.
- Clean HTML/CSS/JavaScript frontend accessible in any modern browser.

The system also handles errors gracefully — invalid symbols, empty datasets, and network failures all return meaningful messages to the user rather than crashing the application.

8.2 Achievements

- Successfully integrated yfinance, scikit-learn, and FastAPI in a working web application.
- Supports both Indian (NSE/BSE) and international (NYSE/NASDAQ) stock symbols.
- Achieved average response times of 3–6 seconds per prediction request.
- No paid services, APIs, or cloud infrastructure required — entirely free and open-source.
- Intuitive UI requiring no technical knowledge to operate.

8.3 Future Scope

- **Advanced Models:** Integrate more sophisticated models such as Random Forest, XGBoost, or ARIMA for better prediction accuracy, especially during high-volatility market periods.
- **Technical Indicators:** Add features like Moving Averages (SMA/EMA), RSI, Bollinger Bands, and MACD to improve the feature set for the model.
- **Multiple Stock Comparison:** Allow users to compare predictions for two or more stocks simultaneously on the same chart.
- **Portfolio Tracking:** Add the ability for users to save a watchlist of stocks and get predictions for all of them at once.
- **Model Evaluation Metrics:** Display accuracy metrics such as MAE and RMSE to help users assess the reliability of predictions.
- **Deployment:** Deploy the application on a cloud platform (Render, Railway, or AWS) so it is accessible publicly without needing to run locally.
- **Historical Accuracy Log:** Track past predictions and compare them with actual prices to display a model accuracy history.
- **Mobile Responsive UI:** Improve CSS for better usability on mobile devices and tablets.
- **Authentication:** Add user accounts so users can save preferences and track their prediction history.

REFERENCES / BIBLIOGRAPHY

1. Research Papers

- Pai, P. F., & Lin, C. S. (2005). A hybrid ARIMA and support vector machines model in stock price forecasting. *Omega*, 33(6), 497–505.
- Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268.
- Nikou, M., Mansourfar, G., & Bagherzadeh, J. (2019). Stock price prediction using DEEP learning algorithm and its comparison with machine learning algorithms. *Intelligent Systems in Accounting, Finance and Management*, 26(4), 164–174.
- Hegazy, O., Soliman, O. S., & Salam, M. A. (2013). A machine learning model for stock market prediction. *International Journal of Computer Science and Telecommunications*, 4(12), 17–23.

2. Websites and Documentation

- FastAPI Official Documentation — <https://fastapi.tiangolo.com>
- yfinance Library Documentation — <https://pypi.org/project/yfinance/>
- scikit-learn Documentation — <https://scikit-learn.org/stable/>
- Chart.js Documentation — <https://www.chartjs.org/docs/latest/>
- Python Official Documentation — <https://docs.python.org/3/>
- Uvicorn Documentation — <https://www.uvicorn.org/>
- Pandas Documentation — <https://pandas.pydata.org/docs/>
- NumPy Documentation — <https://numpy.org/doc/>
- MDN Web Docs (JavaScript Fetch API) — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Yahoo Finance — <https://finance.yahoo.com/>

APPENDICES

Appendix A: Glossary

- **API (Application Programming Interface):** A set of definitions and protocols that allows one software component to communicate with another.
- **FastAPI:** A modern Python web framework for building APIs, known for its speed and automatic interactive documentation.
- **yfinance:** An open-source Python library that provides a convenient interface to download financial data from Yahoo Finance.
- **Linear Regression:** A supervised machine learning algorithm that models the relationship between a dependent variable and one or more independent variables by fitting a linear equation.
- **scikit-learn:** A Python machine learning library providing simple and efficient tools for data mining and analysis.
- **Chart.js:** An open-source JavaScript library for creating responsive, interactive charts in the browser using HTML5 canvas.
- **Uvicorn:** An ASGI-compatible web server for Python, used to serve FastAPI applications.
- **Jinja2:** A Python template engine used to render dynamic HTML pages in FastAPI.
- **Sliding Window:** A technique used to create training samples from time-series data by moving a fixed-size window across the data.
- **OHLCV:** Open, High, Low, Close, Volume — the standard fields in stock market data.

Appendix B: Project File Structure

Stock FINAL/

```
|— app.py
|— requirements.txt
|— templates/
|   └─ index.html
└─ static/
    └─ style.css
    └─ script.js
```

Appendix C: Dependencies (requirements.txt)

```
fastapi
uvicorn
yfinance
pandas
numpy
scikit-learn
jinja2
python-multipart
```

Appendix D: How to Run the Project

- Install Python 3.10 or above from <https://python.org>
- Create a virtual environment: `python -m venv .venv`
- Activate the virtual environment: `.venv\Scripts\activate` (Windows) or `source .venv/bin/activate` (Linux/macOS)
- Install dependencies: `pip install -r requirements.txt`
- Start the server: `uvicorn app:app --reload`
- Open a browser and go to: `http://127.0.0.1:8000`
- Enter a stock symbol (e.g., AAPL, TSLA, RELIANCE.NS) and click Predict.